

KGDB with VirtualBox

Debugging an application live has always been easy for application developers, but debugging a live kernel has never been a simple option for kernel developers—it involves multiple machines with serial connections. This article shows how to use virtualisation atop a running OS to help debug a live kernel on a single machine. Readers are expected to have prior knowledge on how to use GDB, know the fundamentals of the Linux kernel, understand custom compilation, apart from knowing how to use VirtualBox or any other virtualisation software.

KGDB is an amazing Linux kernel debugging tool. It can debug the kernel while it is running, set breakpoints, and step through the code. Earlier, KGDB used to be a bunch of patches that had to be carefully merged into the mainline kernel. However, since version 2.6.26, KGDB has been merged into the mainline, and only needs to be enabled during kernel compilation.

A typical KGDB set-up requires two machines connected by a serial cable: one as a source machine on which debugging is done, and the other (destination) which is being debugged. With virtualisation, however, we can do away with that second machine.

VirtualBox, a virtualisation software originally developed by Innotek and today owned by Oracle, is capable of running operating systems as guests in 'virtual machines'. A guest OS in a VirtualBox VM works pretty much like what it looks like on a normal machine. When we combine VirtualBox with KGDB on a single machine, the host OS is the 'source' machine, while the guest OS (Linux kernel compiled with KGDB enabled) is the 'destination'. A virtual serial port is enabled between the host and the guest. Figure 1 displays such a set-up.

Prerequisites

For this set-up, we'll need:

- The host running a Linux system (you can have a host OS other than Linux, but this article does not cover that). My host system runs Ubuntu Maverick Meerkat 10.10, 64-bit.
- VirtualBox software installed on the host OS. I used the VirtualBox 4.0 distribution-specific binary obtained from Oracle's site, http://www.VirtualBox.org/wiki/Linux_Downloads.
- The *socat* binary installed on the host. This is used to link

the pipe file (FIFO) that is created by VirtualBox, with a pseudo-terminal on the host system. Download it from <http://www.dest-unreach.org/socat/download/>. Normally, GDB takes a physical terminal file (like *ttyS0*) as the remote target, but in our case, we will instead provide a pseudo terminal created by *socat* as the remote target for GDB. Refer to the *socat* man pages for more information.

- A VM installed with a Linux guest OS (I used Fedora 14). The VirtualBox documentation shows how to create a VM, if you need it, so I won't repeat it here. Help on how to install an OS in a VirtualBox VM is also in the documentation. I downloaded the Fedora 14 ISO from the Fedora site, attached it to the VM, and booted the VM and installed Fedora.
- The Linux kernel source is accessible to the VM (and the host, too—see below). This can be picked up from kernel.org; I used version 2.6.37. It is used to re-compile the guest OS kernel with KGDB-specific options. How to get the source available in the VM is described under 'File-sharing between machines' below.

Setting up the guest

Configuring the virtual serial port in VirtualBox

Right-click your virtual machine instance, and go to the *Settings* tab. On the 'Port 1' tab, choose 'Enable Serial Port'. Select 'Port Mode' to be 'Host pipe'. Enter a pipe file-name in the 'Port/File Path' text field, as shown in Figure 2.

File-sharing between machines

I have set up networking for the VM and in my Fedora guest, so that I can easily access files on the host. You could set up an optional NFS server on the host machine, and create an NFS share